Xijing Wang (Thomas)
Team 6
Teammates: Dillon Ma, Mithun Roy, Leena Aljurashi
IRL02 - Motor
Feb 4, 2026

Individual Process

This week, I worked on system integration and interface development for the motor lab, While each motor and sensor pair logical (3 pairs) was implemented by each of my other teammates, my main responsibility was to integrate these independently developed control logics into a single, unified system and to design and implement the GUI for monitoring and commanding the robot.

Mithun was responsible for the hardware setup of the DC motor and implementing the potentiometer based speed control logic. Dillon was responsible for the stepper motor and developed the joystick based control logic. Leena was responsible for the servo motor and developed the IMU based control logic that mapped orientation data to servo motion.

Challenges

One challenge during the motor lab was integrating multiple independently developed motor control logics into a single system. Since each motor was initially developed and tested in isolation, some bugs appear because of conflict. For example, the use of the servo library will disable pins 9 and 10 which have been stated in the canvas, but we figured out in the end. Also, I need to create a scalable and organized system so I created a header file for declaration.

Another challenge is related to GUI design and building reliable communication between the GUI and the microcontroller. Since this is my first robotics project, I have no prior experience in developing a GUI that directly communicates with hardware, even though I have extensive experience building iOS apps, web applications, and personal websites. Therefore, I need to clarify the requirements of a well-designed GUI. Furthermore, I need to design a clear and consistent data format for transmitting motor states and sensor information.

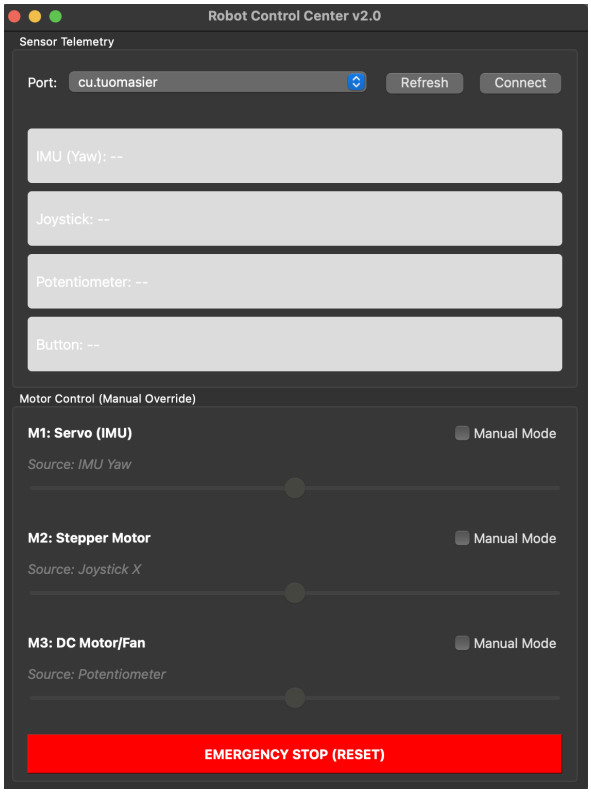
Team Work

We have good weather this week, so all of us are able to join the project from the beginning. We divided responsibilities clearly across hardware implementation, motor-level control logical and system level integration based on our background knowledge. All my other team members have more hardware background like Dillon is studying ME, Mithun and Leena are studying ECE. So, I took the responsibility of all the software parts. The division of labor allowed us to make steady progress on multiple subsystems in parallel while maintaining clear ownership and coordination.

Plans

For the next stage of the project, I will continue to focus on system-level software integration and control logic development. Our team's project will develop a multi-legged robot(hexpot) with coordinated motion behaviors. We are currently exploring hexagon-shaped body design to support symmetric leg placement and stable gait patterns. Each leg will have 2-3 degrees of freedom with a two DOF configuration as a fallback option. On the sensing and control side, we plan to use an IMU to provide orientation and balance feedback, and an ESP32-based microcontroller with bluetooth connectivity to support wireless control and system monitoring. As the prince progresses, we will continue refining the mechanical design, control logic, and system integration, so the plan could change based on the condition and demand.

GUI - Robot Control Panel implemented by PySide6 (Qt for Python)



The GUI was built in Python using PySide6 as a simple control and monitoring interface for the robot. It consists of a sensor display section and a motor control section, all created through code-based layouts without using a visual UI designer. Sensor data is received from the microcontroller over a serial connection and displayed in real time, while motor commands are sent from the GUI using sliders, checkboxes, and an emergency stop button. Communication is handled through a serial interface, allowing the GUI to support both automatic sensor-driven control and manual user override.

IMU logical and servo motor code

```
void updateMotor1(int val) {
 val = constrain(val, 0, 180);
 motor1.write(val);
}
```

Explanation: The servo motor is controlled using yaw data from the BNO055 IMU. The IMU provides orientation measurements in degrees (0–360°), which are continuously read in automatic mode. The yaw angle is linearly mapped to the servo's angular range (0–180°) and sent directly to the servo as a position command. This allows the servo motor to reflect the robot's heading in a smooth and intuitive manner.

Joystick logical and stepper motor code

```
const float STEPS_PER_REV = 4096.0f;
```

```
AccelStepper stepper2(AccelStepper::HALF4WIRE, PIN_M2_IN1, PIN_M2_IN2, PIN_M2_IN3,
PIN_M2_IN4);
int joyX_Center = 512;
int joyY_Center = 512;
const float DEADZONE_NORM = 0.20f;

void calibrateJoystickCenter(int samples = 80) {
 long sx = 0, sy = 0;
 for (int i = 0; i < samples; i++) {
   sx += analogRead(PIN_JOY_X);
   sy += analogRead(PIN_JOY_Y);
   delay(5);
 }
 joyX_Center = (int)(sx / samples);
 joyY_Center = (int)(sy / samples);
}
long degToStepsWithinRev(float deg) {
 while (deg >= 360.0f) deg -= 360.0f;
 while (deg < 0.0f)    deg += 360.0f;
 long spr = (long)lround(STEPS_PER_REV);
 long steps = lround(deg * (STEPS_PER_REV / 360.0f));
 steps %= spr;
 if (steps < 0) steps += spr;
 return steps;
}

long nearestEquivalentTarget(long currentPos, long targetWithinRev) {
 long spr = (long)lround(STEPS_PER_REV);
 long baseRev = currentPos / spr;
 if (currentPos < 0 && (currentPos % spr)) baseRev -= 1;
 long cand = baseRev * spr + targetWithinRev;
 long candMinus = cand - spr;
 long candPlus  = cand + spr;
 long d      = labs(cand      - currentPos);
```

```
long dMinus = labs(candMinus - currentPos);
long dPlus  = labs(candPlus  - currentPos);
if (dMinus < d) { cand = candMinus; d = dMinus; }
if (dPlus  < d) { cand = candPlus; }
return cand;
}
void updateMotor2_Auto(int rawX, int rawY) {
int dx = rawX - joyX_Center;
int dy = rawY - joyY_Center;
float x = (float)dx / 512.0f;
float y = (float)dy / 512.0f;
float r = sqrtf(x*x + y*y);
if (r < DEADZONE_NORM) return;
float angDeg = atan2f(y, x) * 180.0f / (float)M_PI;
if (angDeg < 0) angDeg += 360.0f;
long targetWithinRev = degToStepsWithinRev(angDeg);
long targetAbs = nearestEquivalentTarget(stepper2.currentPosition(), targetWithinRev);
stepper2.moveTo(targetAbs);
}
```

Explanation: The stepper motor is controlled by a two-axis joystick. Raw joystick readings are first centered using a calibration procedure and normalized relative to the joystick's neutral position. A dead zone is applied to avoid unintended motion near the center. The joystick direction is then converted into a target rotation angle using polar coordinates. This angle is mapped to an equivalent step position within one revolution, and the closest absolute target position is selected to minimize unnecessary rotation. The stepper motor is updated continuously in each control loop to ensure smooth and responsive motion.

Potentiometer and DC motor code

```
// --- Motor 3 (DC Motor) <--- Potentiometer ---
if (!manualMode[2]) {
int pwmSpeed = 0;
if (data.potVal > 20) {
  pwmSpeed = map(data.potVal, 20, 1023, 60, 255);
}
updateMotor3(pwmSpeed);
```

```
}
void updateMotor3(int pwmSpeed) {
 pwmSpeed = constrain(pwmSpeed, 0, 255);
 digitalWrite(PIN_M3_IN1, HIGH);
 digitalWrite(PIN_M3_IN2, LOW);
 analogWrite(PIN_M3_ENABLE, pwmSpeed);
}
```

Explanation: The DC motor speed is controlled by a potentiometer input. The raw analog value from the potentiometer is read and mapped to a PWM speed command. A small threshold is applied to prevent motor activation at very low input values. The motor direction is fixed, while the enable pin receives a PWM signal to regulate speed. This provides a simple and intuitive continuous speed control for the DC motor.